



Universidad de Murcia

FACULTAD DE INFORMÁTICA

Práctica 2

UDP AirMonitor

Programación para las Comunicaciones

Junio 2025/2026

Autor:

María Ballester Martínez
DNI: 48853989B
m.ballestermartinez@um.es

Docente:

Humberto Martínez Barbera
humberto@um.es

Índice

1	Introducción	2
2	Funcionalidades	2
2.1	Funcionalidades obligatorias	2
2.2	Funcionalidades opcionales	2
2.3	Arquitectura de servidores y clientes	2
2.4	Modelo de validación	3
2.5	Configuración de direcciones IP y puertos	3
3	Implementación	3
3.1	Protocolo y formato de trama	3
3.2	Operaciones de E/S e hilos	4
3.3	Modelos de serialización	4
3.4	Organización del código	4
4	Ejecución	5

1 Introducción

Esta práctica implementa una aplicación cliente-servidor de distribución de información *de N a M* sobre un servicio de transporte no orientado a conexión (*UDP*). El escenario simulado es un sistema distribuido de **gestión de la calidad del aire**: un conjunto de servidores difunde periódicamente distintas medidas (temperatura, humedad, PM10, SO₂, NO₂, O₃) y los clientes las reciben y muestran.

Se manejan dos tipos de mensaje: los **mensajes de difusión**, enviados por los servidores en modo *broadcast*, y los **mensajes de control**, enviados por los clientes en modo *unicast* a un servidor concreto, el cual responde únicamente a dicho cliente. Toda la información se serializa indistintamente en **XML** (mediante DOM) o **JSON** (mediante GSON). Los documentos XML se **validan explícitamente** contra un esquema XML Schema en la recepción, y todas las tramas transmitidas y recibidas se almacenan como trazas en el directorio */logs* para su posterior verificación.

2 Funcionalidades

2.1 Funcionalidades obligatorias

- **Difusión periódica** (broadcast) de tramas con un mínimo de tres variables, con su valor actual y sus unidades. El cliente muestra cada trama recibida indicando el servidor emisor y el formato de llegada.
- **Mensajes de control** (unicast) dirigidos a un servidor. Se implementan cuatro operaciones: cambio de codificación (XML/JSON) en tiempo de ejecución, cambio de frecuencia de transmisión, cambio de unidades de una variable y activación/desactivación del envío. El servidor responde sólo al cliente que lo solicitó.
- **Doble serialización** XML (DOM) y JSON (GSON). La cabecera del protocolo especifica el formato empleado.
- **Validación** explícita de los mensajes XML contra XML Schema en el momento de la recepción, con notificación por consola.
- **Almacenamiento de trazas** de la información transmitida y recibida (XML/JSON) en */logs*.

2.2 Funcionalidades opcionales

- **Gestión de pérdidas con reenvíos**: el cliente reintenta el mensaje de control (hasta tres veces) si no recibe respuesta dentro del tiempo de espera.
- **Multi-cliente simultáneo** en el mismo equipo, habilitando *SO_REUSEADDR* en el socket de difusión.
- **Difusión por broadcast real** a 255.255.255.255 (dirección configurable), en lugar de un simple *unicast* a *localhost*.
- **Varios tipos de trama de control**.

2.3 Arquitectura de servidores y clientes

La arquitectura del **servidor es multi-hilo**, y se justifica porque combina dos responsabilidades concurrentes de naturaleza distinta: la difusión es una tarea *periódica* (un productor que emite cada *T* segundos), mientras que la atención de control es *reactiva* y *bloqueante* (espera la llegada de datagramas). Con un único hilo, la llamada bloqueante de recepción de control retrasaría la difusión, o bien obligaría a acoplar ambas cadencias mediante temporizadores; por ello cada servidor emplea **dos hilos** (difusión y control).

El **cliente** usa un **hilo demonio** dedicado a la recepción continua de mensajes de difusión (bloqueante en *receive()*) y el **hilo principal** para la interacción con el usuario y el envío de

control. Esta separación es la más idónea porque la recepción de difusión es asíncrona respecto a la actividad del menú, y permite seguir mostrando tramas mientras el usuario escribe un comando.

2.4 Modelo de validación

Se ha optado por **XML Schema (XSD)** frente a DTD porque permite tipar los datos (`xs:double`, `xs:long`), restringir la cardinalidad (se exige un mínimo de tres variables con `minOccurs="3"`) y enumerar los valores válidos del tipo de control. La validación se realiza **en recepción** y no en emisión, ya que su objetivo es proteger al receptor frente a datos mal formados o no válidos que pudieran llegar por el canal; al superarse, se escribe en consola "Mensaje XML validado correctamente".

2.5 Configuración de direcciones IP y puertos

Servicio	Dirección	Puerto	Carácter
Difusión (broadcast)	255.255.255.255	5000	Puerto fijo; dirección configurable
Control servidor i	host del servidor	6000+ i	Fijo (6001–6003)
Socket control cliente	host del cliente	Variable	Lo asigna el S.O.
Identificador de cliente	—	—	Variable

Tabla 1: Configuración de direcciones y puertos.

3 Implementación

3.1 Protocolo y formato de trama

Cada datagrama UDP se compone de una **cabecera de 2 bytes** seguida de la **carga útil** serializada en UTF-8. El primer byte indica el tipo de mensaje y el segundo el formato de serialización, de modo que el receptor sabe a qué clase deserializar y qué esquema usar para validar.

Tipo (1 byte) 'D' / 'C'	Formato (1 byte) 'X' / 'J'	Carga útil: documento XML o JSON (UTF-8)
----------------------------	-------------------------------	--

Figura 1: Formato de trama del protocolo de aplicación.

La figura siguiente resume el intercambio en los dos modos de comunicación.

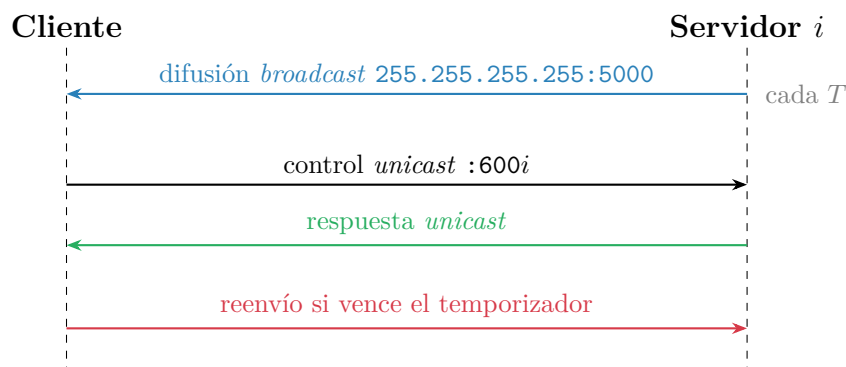


Figura 2: Diagrama de secuencia: difusión (broadcast) y control (unicast con reenvíos).

3.2 Operaciones de E/S e hilos

La E/S se realiza con **sockets UDP** (`DatagramSocket` y `DatagramPacket`). Se emplea un modelo de **E/S bloqueante con un hilo por flujo**: es el más sencillo y suficiente dado el número reducido y fijo de flujos, por lo que no se justifica el uso de E/S no bloqueante (NIO) ni multiplexación con selectores. El socket de control del cliente fija un *timeout* (`setSoTimeout`) que permite implementar los reenvíos sin bloquearse indefinidamente. La difusión utiliza un socket con `setBroadcast(true)` y el receptor de difusión del cliente `setReuseAddress(true)` para admitir varios clientes.

Hilo	Entidad	Función
<code>difusion-srv<i>i</i></code>	Servidor	Genera y difunde una trama cada periodo
<code>control-srv<i>i</i></code>	Servidor	Recibe control, lo aplica y responde
<code>difusion-cliente-<i>x</i></code>	Cliente	Recibe y muestra las tramas de difusión
<code>main</code>	Cliente	Menú, envío de control y espera de respuesta

Tabla 2: Hilos implementados.

3.3 Modelos de serialización

El *marshalling/unmarshalling* se abstrae tras la interfaz `ISerializador` (métodos `serializar()` y `deserializar()`), con dos implementaciones seleccionables mediante `SerializadorFactory`:

- **XML con DOM**: en el *marshalling* se construye un `Document` con `DocumentBuilder`, se añaden los elementos y se vuelca a bytes con un `Transformer`. En el *unmarshalling* se parsea el documento, se **valida** con un `Validator` contra el *XSD* y se recorre el *árbol DOM* para reconstruir el objeto.
- **JSON con GSON**: `gson.toJson()` y `gson.fromJson()` sobre las clases del modelo.

Se definen dos esquemas: `distribucion.xsd` (estructura del mensaje de difusión, con `minOccurs="3"` en las variables) y `control.xsd` (que restringe el tipo de control a una enumeración). La carga útil es idéntica en contenido; sólo cambia su representación (XML a la izquierda, JSON a la derecha, abreviados):

1 XML	JSON
2 <distribucion>	{
3 <idServidor>srv1</idServidor>	"idServidor": "srv1",
4 <marcaTiempo>1780788361</marcaTiempo>	"marcaTiempo": 1780788361,
5 <variables>	"variables": [
6 <variable>	{ "nombre": "temperatura",
7 <nombre>temperatura</nombre>	"valor": 21.3,
8 <valor>21.3</valor>	"unidad": "C" }, ...
9 <unidad>C</unidad>	]
10 </variable> ...	}
11 </variables>	
12 </distribucion>	

3.4 Organización del código

El código se estructura en una **jerarquía de clases** encabezada por la clase abstracta `Mensaje`, de la que derivan los dos tipos de mensaje. La serialización y la red se desacoplan en paquetes independientes.

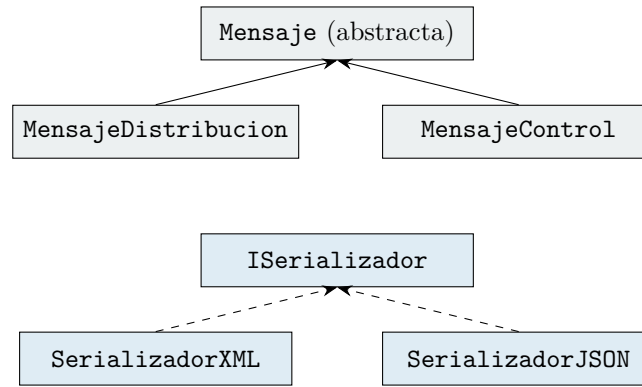


Figura 3: Diagrama de clases: jerarquía de mensajes e interfaz de serialización.

Paquete	Clases
modelo	Mensaje, MensajeDistribucion, MensajeControl, Variable, TipoMensaje, TipoControl
serializacion	ISerializador, SerializadorXML, SerializadorJSON, SerializadorFactory, Formato, SerializacionException
red	Protocolo, Cabecera, Empaquetador, AlmacenDatos, Servidor, Cliente
app	MainServidor, MainServidores, MainCliente

Tabla 3: Paquetes y clases implementadas.

4 Ejecución

El proyecto es un proyecto **Maven**; la dependencia GSON se declara en el `pom.xml` y se resuelve automáticamente. Una vez importado el proyecto en Eclipse, basta con dos ejecuciones:

- `ppc.java.app.MainServidores`: arranca los tres servidores (cada uno en sus hilos) dentro de una misma JVM.
- `ppc.java.app.MainCliente`: cliente interactivo que recibe la difusión y permite enviar mensajes de control.

Para un despliegue distribuido en varias máquinas puede usarse `MainServidor` con el número de servidor (1–3) como argumento. Las trazas se escriben en `/logs`.

Una vez ejecutados los servidores y el cliente veremos que empiezan a aparecer, cada poco tiempo, en la terminal del cliente los mensajes que mandan los servidores. Esto es lo que podemos visualizar en la figura 4.

```

>
--- Trama recibida (XML) - Mensaje XML validado correctamente ---
[DISTRIBUCION] servidor=srv3 t=1780871576598
  O3 = 159,47 ug/m3
  humedad = 68,72 %
  PM10 = 23,06 ug/m3
>
--- Trama recibida (JSON) ---
[DISTRIBUCION] servidor=srv1 t=1780871576598
  temperatura = 11,10 C
  humedad = 59,33 %
  PM10 = 136,74 ug/m3
  
```

Figura 4: Tramas de difusión recibidas (XML y JSON).

Además, en el cliente, al iniciarlo nos muestra un menu que nos dice:

```
1 === MENU CLIENTE (cli1) ===
2 1) Enviar mensaje de control
3 2) Salir
```

Si escribimos el 1 nos mostrara otro menu. En este podremos configurar el mensaje de control que enviará el cliente al servidor. Dicho menu, un ejemplo del funcionamiento y la respuesta del servidor es lo que podemos observar en la siguiente figura:

```
> 1
Servidor destino (1..3): 1
Tipo de control:
  1) Cambiar codificacion (XML/JSON)
  2) Cambiar frecuencia (Hz)
  3) Cambiar unidades de una variable
  4) Activar / Desactivar envio
> 1
Nueva codificacion (XML/JSON): JSON
Formato de la peticion (XML/JSON) [XML]: JSON
[cli1] Control enviado a localhost:6001 (JSON) [intento 1/3]
RESPUESTA del servidor: OK: codificacion cambiada a JSON
```

Figura 5: Menú de control y respuesta del servidor.

Tras enviar este mensaje de control en la terminal del servidor veremos un mensaje, que deja claro quien lo envia y que acción quiere que haga:

```
1 [srv1] Control recibido de 127.0.0.1:64840 -> [CONTROL] cliente=cli1 servidor=
  srv1 tipo=CAMBIO_CODIFICACION valor=JSON
```